AI Time Table Generator Using Python, Streamlit and Genetic Algorithm

Submitted By

Name: _Ubaidullah Khan and Malik Talal_

Roll Number: _FA24-BAI-052 ,FA24-BAI-054_

Semester: _4th_

Department: _CS_

Institute: _Comsats University Islamabad_

Abstract

The AI Time Table Generator is an intelligent scheduling system developed using Python and Streamlit. The project automatically creates optimized and clash-free timetables using a Genetic Algorithm.

Manual timetable generation is difficult because teachers, rooms, classes, and subjects must all be arranged without conflicts. This project automates the process and reduces human effort.

The system uses Artificial Intelligence concepts, Data Structures, and optimization techniques to generate efficient schedules.

Introduction

Timetable scheduling is one of the most important tasks in educational institutions. Creating schedules manually often causes problems such as:

- Teacher clashes
- Room conflicts
- Unbalanced schedules
- Time wastage
- Human errors

This project solves these problems using Artificial Intelligence.

The system generates automatic timetables based on user inputs such as:

- Subjects
- Teachers
- Rooms
- Days
- Time slots

The project uses a Genetic Algorithm to continuously improve timetable quality.

Objectives

The objectives of this project are:

- Generate automatic timetables
- Eliminate teacher clashes
- Eliminate room conflicts
- Optimize scheduling
- Reduce manual work
- Implement Artificial Intelligence techniques
- Create an interactive Streamlit GUI
- Store data permanently using SQLite

Technologies Used

Technology	Purpose
Python	Main programming language
Streamlit	Frontend GUI
SQLite	Database storage
Pandas	Data display
Random Module	Random timetable generation
Genetic Algorithm	AI optimization

Data Structures Used

Lists

Used to store:

- Subjects
- Days
- Rooms
- Time slots

Dictionaries

Used to store:

- Teacher mapping
- Timetable data

Nested Dictionaries

Used to represent timetable structure.

Example:

Day → Time Slot → Lecture Details

System Architecture

Frontend

The frontend is developed using Streamlit.

Users can:

- Enter subjects
- Enter teachers
- Enter rooms
- Generate timetable
- View generated timetable

Backend

The backend is implemented in Python.

It handles:

- Timetable generation
- Clash detection
- Fitness evaluation
- Genetic Algorithm operations

Database

SQLite database stores:

- Teachers
- Subjects
- Rooms
- Timetables

This allows permanent data storage.

Genetic Algorithm

The project uses a Genetic Algorithm because timetable scheduling is a Constraint Satisfaction Problem.

The algorithm works through multiple generations and continuously improves the timetable.

Working of Genetic Algorithm

Step 1 — Population Creation

Generate multiple random timetables.

Step 2 — Fitness Function

Evaluate timetable quality.

Step 3 — Selection

Choose best timetables.

Step 4 — Crossover

Combine good timetables.

Step 5 — Mutation

Apply small random changes.

Step 6 — Repeat

Repeat process until optimized timetable is achieved.

Fitness Function

The fitness function evaluates timetable quality.

Formula:

Fitness = 100 − (Clashes × 10)

Higher fitness means:

- Better timetable
- Fewer conflicts
- More optimization

Constraints Used

The system checks the following constraints:

- Teacher clash detection
- Room clash detection
- Duplicate lecture checking
- Subject frequency management
- Lab room allocation
- Break timing management

Streamlit GUI

The project uses Streamlit for GUI development.

Features:

- User-friendly interface
- Interactive forms
- Timetable generation button
- Fitness score display
- Dynamic timetable table

Project Workflow

Step 1

User enters timetable information.

Step 2

Frontend sends data to backend.

Step 3

Python backend generates random schedules.

Step 4

Genetic Algorithm optimizes timetable.

Step 5

Best timetable is selected.

Step 6

Final timetable is displayed in Streamlit.

Step 7

Data is saved into SQLite database.

Advantages of the System

- Saves time
- Reduces manual effort
- Eliminates clashes
- Generates optimized schedules
- Easy to use
- Scalable system
- Intelligent scheduling

Limitations

- Basic version handles limited constraints
- Large institutions may require advanced optimization
- Requires proper input formatting

Future Enhancements

Future improvements may include:

- Teacher preference system
- Multiple department support
- Exam timetable generation
- Attendance integration
- PDF export
- Email notifications
- Cloud database support
- Login system
- Admin dashboard

Results

The system successfully generates optimized and clash-free timetables.

The Genetic Algorithm improves timetable quality generation by generation.

The Streamlit GUI makes the system interactive and user-friendly.

Conclusion

The AI Time Table Generator demonstrates how Artificial Intelligence can solve real-world scheduling problems efficiently.

Using Python, Streamlit, SQLite, and Genetic Algorithms, the system automatically creates optimized timetables while minimizing conflicts.

The project combines:

- Artificial Intelligence
- Data Structures
- GUI development
- Database integration
- Optimization techniques

This makes the project practical, scalable, and suitable for educational institutions.

Final Project Statement

“This project uses Artificial Intelligence and Genetic Algorithm techniques to automatically generate optimized and clash-free academic timetables.”
